

ECS 409/609

Assignment 1

Sequential Construct-I

Integer Arithmetic and Logical Instructions

Kunwar Arpit Singh

Submission Deadline: August 28, 2025

For code repository of this assignment:  **MIPS Assignments**

1. Problem 1

Problem statement

For a given arithmetic and geometric progression sequence:

- 1, 11, 21, 31, ... (A.P.)
- 4, 8, 16, ... (G.P.)

Write a MIPS assembly program to compute the eighth term (in A.P.) / fourth term (in G.P.) and sum of the first six terms (in A.P.) / first four numbers (in G.P.). The initial values are stored in registers or memory.

Part A: Arithmetic Progression

MIPS Assembly Source

```
1  .data
2      myName: .asciiz "Kunwar_Arpit_Singh\n\n"
3      term_msg: .asciiz "The_8th_term_is:"
4      sum_msg: .asciiz "\nThe_sum_of_the_first_6_terms_is:"
5      newline: .asciiz "\n"
6
7  .text
8  .globl main
9
10 main:
11      li $v0, 4
12      la $a0, myName
13      syscall
14
15      # $t0 = a
16      li $t0, 1
17
18      # $t1 = d
```

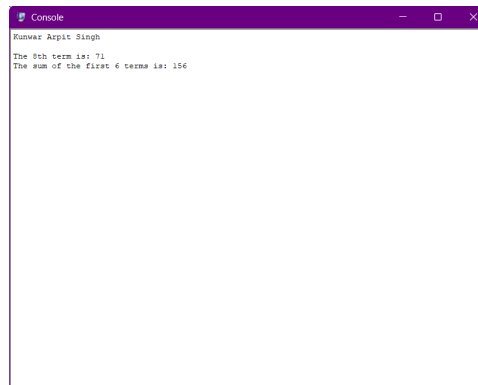
```

19      li $t1, 10
20
21      # $t2 = n
22      li $t2, 8
23
24      # Calculate the 8th term (n=8)
25
26      addi $t3, $t2, -1
27
28      mul $t4, $t3, $t1
29
30      add $s0, $t0, $t4
31
32      # Calculate the sum of the first 6 terms (n=6)
33
34      li $t2, 6
35
36      addi $t3, $t2, -1
37
38      mul $t4, $t3, $t1
39
40      mul $t5, $t0, 2
41
42      add $t6, $t5, $t4
43
44      li $t8, 2
45      div $t2, $t8
46
47      mflo $t7
48
49      mul $s1, $t7, $t6
50
51      li $v0, 4
52      la $a0, term_msg
53      syscall
54
55      li $v0, 1
56      move $a0, $s0
57      syscall
58
59      li $v0, 4
60      la $a0, sum_msg
61      syscall
62
63      li $v0, 1
64      move $a0, $s1
65      syscall
66
67      li $v0, 10
68      syscall

```

Output Screenshots

Console output (A.P.)



Register values (A.P.)

Int Regs [16]		
PC	=	4000ac
EPC	=	0
Cause	=	0
BadVAddr	=	0
Status	=	3000ff10
HI	=	0
LO	=	9c
R0	[r0]	= 0
R1	[at]	= 10010000
R2	[v0]	= a
R3	[v1]	= 0
R4	[a0]	= 9c
R5	[a1]	= 7ffffef20
R6	[a2]	= 7ffffef30
R7	[a3]	= 0
R8	[t0]	= 1
R9	[t1]	= a
R10	[t2]	= 6
R11	[t3]	= 5
R12	[t4]	= 32
R13	[t5]	= 2
R14	[t6]	= 34
R15	[t7]	= 3
R16	[s0]	= 47
R17	[s1]	= 9c
R18	[s2]	= 0
R19	[s3]	= 0
R20	[s4]	= 0
R21	[s5]	= 0
R22	[s6]	= 0
R23	[s7]	= 0
R24	[t8]	= 2
R25	[t9]	= 0
R26	[k0]	= 0
R27	[k1]	= 0
R28	[gp]	= 10008000
R29	[sp]	= 7ffffef1c
R30	[s8]	= 0
R31	[ra]	= 400018

Part B: Geometric Progression

MIPS Assembly Source

```
1 .data
2     myName: .asciiz "Kunwar_Arpit_Singh\n\n"
3     term_msg: .asciiz "The_4th_term_is:"
4     sum_msg: .asciiz "\nThe_sum_of_the_first_4_terms_is:"
5
6 .text
7 .globl main
8
9 main:
10     li $v0, 4
11     la $a0, myName
12     syscall
13
14     # $t0 = a
15     li $t0, 4
16
17     # $t1 = r
18     li $t1, 2
19
20     # $t2 = n
21     li $t2, 4
22
23     mul $t3, $t1, $t1
24     mul $t3, $t3, $t1
25
26     mul $s2, $t0, $t3
27
28     # Calculate the sum of the first 4 terms (n=4)
29
30     mul $t4, $t3, $t1
31     addi $t5, $t4, -1
32     mul $t6, $t0, $t5
33     addi $t7, $t1, -1
34
35     div $t6, $t7
36     mflo $s3
37
38     li $v0, 4
39     la $a0, term_msg
40     syscall
41
42     li $v0, 1
43     move $a0, $s2
44     syscall
45
46     li $v0, 4
47     la $a0, sum_msg
48     syscall
49
```

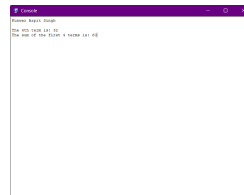
```

50      li $v0, 1
51      move $a0, $s3
52      syscall
53
54      li $v0, 10
55      syscall

```

Output Screenshots

Console output (G.P.)



Register values (G.P.)

Int Regs [16]	
PC	= 40009c
EPC	= 0
Cause	= 0
BadVAddr	= 0
Status	= 3000ff10
HI	= 0
LO	= 3c
R0 [r0]	= 0
R1 [at]	= 10010000
R2 [v0]	= a
R3 [v1]	= 0
R4 [a0]	= 3c
R5 [a1]	= 7ffffef20
R6 [a2]	= 7ffffef30
R7 [a3]	= 0
R8 [t0]	= 4
R9 [t1]	= 2
R10 [t2]	= 4
R11 [t3]	= 8
R12 [t4]	= 10
R13 [t5]	= f
R14 [t6]	= 3c
R15 [t7]	= 1
R16 [s0]	= 0
R17 [s1]	= 0
R18 [s2]	= 20
R19 [s3]	= 3c
R20 [s4]	= 0
R21 [s5]	= 0
R22 [s6]	= 0
R23 [s7]	= 0
R24 [t8]	= 0
R25 [t9]	= 0
R26 [k0]	= 0
R27 [k1]	= 0
R28 [gp]	= 10008000
R29 [sp]	= 7ffffef1c
R30 [s8]	= 0
R31 [ra]	= 400018

2. Problem 2

Problem statement

Write a MIPS assembly program to implement a half adder and a half subtractor using logical instructions.

MIPS Assembly Source

```
1  .data
2      myName: .asciiz "Kunwar_Arpit_Singh\n"
3      inputs_00: .asciiz "\nInputs: A=0, B=0"
4      inputs_01: .asciiz "\n\nInputs: A=0, B=1"
5      inputs_10: .asciiz "\n\nInputs: A=1, B=0"
6      inputs_11: .asciiz "\n\nInputs: A=1, B=1"
7
8      adder_title: .asciiz "\nHalf Adder Results:"
9      sum_msg: .asciiz "\nSum:_"
10     carry_msg: .asciiz "\nCarry:_"
11     sub_title: .asciiz "\nHalf Subtractor Results"
12     diff_msg: .asciiz "\nDifference:_"
13     borrow_msg: .asciiz "\nBorrow:_"
14
15  .text
16  .globl main
17
18  main:
19      # Print name
20      li $v0, 4
21      la $a0, myName
22      syscall
23      li $t0, 0
24      li $t1, 0
25      li $v0, 4
26      la $a0, inputs_00
27      syscall
28      # Calculations
29      xor $s0, $t0, $t1
30      and $s1, $t0, $t1
31      xor $s2, $t0, $t1
32      not $t5, $t0
33      and $s3, $t5, $t1
34      jal PrintResults
35      li $t0, 0
36      li $t1, 1
37      li $v0, 4
38      la $a0, inputs_01
39      syscall
40      xor $s0, $t0, $t1
41      and $s1, $t0, $t1
42      xor $s2, $t0, $t1
```

```

43     not $t5, $t0
44     and $s3, $t5, $t1
45     jal PrintResults
46     li $t0, 1
47     li $t1, 0
48     li $v0, 4
49     la $a0, inputs_10
50     syscall
51     xor $s0, $t0, $t1
52     and $s1, $t0, $t1
53     xor $s2, $t0, $t1
54     not $t5, $t0
55     and $s3, $t5, $t1
56     jal PrintResults
57     li $t0, 1
58     li $t1, 1
59     li $v0, 4
60     la $a0, inputs_11
61     syscall
62     xor $s0, $t0, $t1
63     and $s1, $t0, $t1
64     xor $s2, $t0, $t1
65     not $t5, $t0
66     and $s3, $t5, $t1
67     jal PrintResults
68     li $v0, 10
69     syscall
70 PrintResults:
71     li $v0, 4; la $a0, adder_title; syscall
72     li $v0, 4; la $a0, sum_msg; syscall
73     li $v0, 1; move $a0, $s0; syscall
74     li $v0, 4; la $a0, carry_msg; syscall
75     li $v0, 1; move $a0, $s1; syscall
76
77     li $v0, 4; la $a0, sub_title; syscall
78     li $v0, 4; la $a0, diff_msg; syscall
79     li $v0, 1; move $a0, $s2; syscall
80     li $v0, 4; la $a0, borrow_msg; syscall
81     li $v0, 1; move $a0, $s3; syscall
82     jr $ra

```

Output Screenshots

Console output (Half Adder/Subtractor)

```

Console
Runway Aspin Singh

Inputs: A=0, B=0
Half Adder Results:
Sum: 0
Carry: 0
Half Subtractor Results:
Difference: 0
Borrow: 0

Inputs: A=0, B=1
Half Adder Results:
Sum: 0
Carry: 1
Half Subtractor Results:
Difference: 1
Borrow: 1

Inputs: A=1, B=0
Half Adder Results:
Sum: 1
Carry: 0
Half Subtractor Results:
Difference: 1
Borrow: 0

Inputs: A=1, B=1
Half Adder Results:
Sum: 0
Carry: 1
Half Subtractor Results:
Difference: 0
Borrow: 0

```

Register values (Half Adder/Subtractor)

Int Regs [16]	
PC	= 400114
EPC	= 0
Cause	= 0
BadVAddr	= 0
Status	= 3000ff10
HI	= 0
LO	= 0
R0 [r0]	= 0
R1 [at]	= 10010000
R2 [v0]	= a
R3 [v1]	= 0
R4 [a0]	= 0
R5 [a1]	= 7ffffef20
R6 [a2]	= 7ffffef30
R7 [a3]	= 0
R8 [t0]	= 1
R9 [t1]	= 1
R10 [t2]	= 0
R11 [t3]	= 0
R12 [t4]	= 0
R13 [t5]	= ffffffff
R14 [t6]	= 0
R15 [t7]	= 0
R16 [s0]	= 0
R17 [s1]	= 1
R18 [s2]	= 0
R19 [s3]	= 0
R20 [s4]	= 0
R21 [s5]	= 0
R22 [s6]	= 0
R23 [s7]	= 0
R24 [t8]	= 0
R25 [t9]	= 0
R26 [k0]	= 0
R27 [k1]	= 0
R28 [gp]	= 10008000
R29 [sp]	= 7ffffef1c
R30 [s8]	= 0
R31 [ra]	= 400110

3. Problem 3

Problem statement

Write a MIPS assembly program to swap the values of two registers using only logical instructions.

MIPS Assembly Source

```
1  .data
2      myName: .asciiz "Kunwar_Arpit_Singh\n\n"
3      before_msg: .asciiz "Before_Swap:_A=_ "
4      after_msg: .asciiz "\nAfter_Swap: _A=_ "
5      b_val_msg: .asciiz ",_B=_ "
6      header1: .asciiz "\nSwap_1:_Positive_Integers:\n"
7      header2: .asciiz "\nSwap_2:_A_is_zero:\n"
8      header3: .asciiz "\nSwap_3:_B_is_zero:\n"
9      header4: .asciiz "\nSwap_4:_Negative_Integers:\n"
10     header5: .asciiz "\nSwap_5:_Mixed_Signs:\n"
11     header6: .asciiz "\nSwap_6:_Identical_Values:\n"
12  .text
13  .globl main
14
15  main:
16      li $v0, 4
17      la $a0, myName
18      syscall
19
20      li $s0, 12345 # A
21      li $s1, 98765 # B
22      la $a1, header1
23      jal RunSingleSwap
24
25      li $s0, 0
26      li $s1, 55555
27      la $a1, header2
28      jal RunSingleSwap
29
30      li $s0, 44444
31      li $s1, 0
32      la $a1, header3
33      jal RunSingleSwap
34
35      li $s0, -100
36      li $s1, -250
37      la $a1, header4
38      jal RunSingleSwap
39
40      li $s0, 789
41      li $s1, -123
42      la $a1, header5
```

```

43     jal RunSingleSwap
44
45     li $s0, 999
46     li $s1, 999
47     la $a1, header6
48     jal RunSingleSwap
49
50     li $v0, 10
51     syscall
52
53 RunSingleSwap:
54     addi $sp, $sp, -4
55     sw $ra, 0($sp)
56     li $v0, 4
57     move $a0, $a1
58     syscall
59     li $v0, 4
60     la $a0, before_msg
61     syscall
62     jal PrintState
63     and $t0, $s0, $s1
64     nor $t1, $t0, $t0
65     or $t2, $s0, $s1
66     and $s0, $t2, $t1
67     and $t0, $s0, $s1
68     nor $t1, $t0, $t0
69     or $t2, $s0, $s1
70     and $s1, $t2, $t1
71     and $t0, $s0, $s1
72     nor $t1, $t0, $t0
73     or $t2, $s0, $s1
74     and $s0, $t2, $t1
75     li $v0, 4
76     la $a0, after_msg
77     syscall
78     jal PrintState
79     lw $ra, 0($sp)
80     addi $sp, $sp, 4
81     jr $ra
82
83 PrintState:
84     li $v0, 1
85     move $a0, $s0
86     syscall
87     li $v0, 4
88     la $a0, b_val_msg
89     syscall
90     li $v0, 1
91     move $a0, $s1
92     syscall
93     jr $ra

```

Output Screenshots

Console output (Register Swap)

```

Console
Runes: Asipit 11/11/2023

Swap 1: Positive Integers:
Before Swap: A = 12345, B = 67890
After Swap: A = 67890, B = 12345
Swap 2: A is zero:
Before Swap: A = 0, B = 16166
After Swap: A = 16166, B = 0
Swap 3: B is zero:
Before Swap: A = 16166, B = 0
After Swap: A = 0, B = 16166
Swap 4: Negative Integers:
Before Swap: A = -100, B = -200
After Swap: A = -200, B = -100
Swap 5: Round Trip:
Before Swap: A = 789, B = -123
After Swap: A = -123, B = 789
Swap 6: Identical Values:
Before Swap: A = 333, B = 333
After Swap: A = 333, B = 333

```

Register values (Register Swap)

Int Regs [16]		
PC	=	4000bc
EPC	=	0
Cause	=	0
BadVAddr	=	0
Status	=	3000ff10
HI	=	0
LO	=	0
R0 [r0]	=	0
R1 [at]	=	10010000
R2 [v0]	=	a
R3 [v1]	=	0
R4 [a0]	=	3e7
R5 [a1]	=	100100bc
R6 [a2]	=	7ffffef30
R7 [a3]	=	0
R8 [t0]	=	0
R9 [t1]	=	ffffffff
R10 [t2]	=	3e7
R11 [t3]	=	0
R12 [t4]	=	0
R13 [t5]	=	0
R14 [t6]	=	0
R15 [t7]	=	0
R16 [s0]	=	3e7
R17 [s1]	=	3e7
R18 [s2]	=	0
R19 [s3]	=	0
R20 [s4]	=	0
R21 [s5]	=	0
R22 [s6]	=	0
R23 [s7]	=	0
R24 [t8]	=	0
R25 [t9]	=	0
R26 [k0]	=	0
R27 [k1]	=	0
R28 [gp]	=	10008000
R29 [sp]	=	7ffffef1c
R30 [s8]	=	0
R31 [ra]	=	4000b8

4. Problem 4

Problem statement

Write a MIPS assembly program to compute a weighted average of four 32-bit numbers (A, B, C, and D) using the formula: $(0.125A + 0.25B + 0.5C + 0.5D)$. Do not use multiplication, division, or floating-point instructions. The final value should be returned in register \$a0.

MIPS Assembly Source

```
1  .data
2      author_msg: .asciiz "Kunwar_Arpit_Singh\n"
3      result_msg: .asciiz ",_The_weighted_average_is:_\n"
4      val_A_msg: .asciiz "\n_A=_\n"
5      val_B_msg: .asciiz ",_B=_\n"
6      val_C_msg: .asciiz ",_C=_\n"
7      val_D_msg: .asciiz ",_D=_\n"
8
9  .text
10 .globl main
11
12 main:
13     li $v0, 4
14     la $a0, author_msg
15     syscall
16
17     li $a0, 800
18     li $a1, 400
19     li $a2, 200
20     li $a3, 100
21     jal calculate_and_print
22
23     li $a0, 160
24     li $a1, 160
25     li $a2, 160
26     li $a3, 160
27     jal calculate_and_print
28
29     li $a0, 64
30     li $a1, 32
31     li $a2, 16
32     li $a3, 8
33     jal calculate_and_print
34
35     li $a0, 80
36     li $a1, 40
37     li $a2, 0
38     li $a3, 0
39     jal calculate_and_print
40
```

```

41      li $a0, 1000
42      li $a1, 500
43      li $a2, 250
44      li $a3, 120
45      jal calculate_and_print
46
47      li $v0, 10
48      syscall
49
50 calculate_and_print:
51
52      move $s0, $a0
53      move $s1, $a1
54      move $s2, $a2
55      move $s3, $a3
56
57
58      li $v0, 4
59      la $a0, val_A_msg
60      syscall
61
62      li $v0, 1
63      move $a0, $s0
64      syscall
65
66
67      li $v0, 4
68      la $a0, val_B_msg
69      syscall
70
71      li $v0, 1
72      move $a0, $s1
73      syscall
74
75      li $v0, 4
76      la $a0, val_C_msg
77      syscall
78
79      li $v0, 1
80      move $a0, $s2
81      syscall
82
83      li $v0, 4
84      la $a0, val_D_msg
85      syscall
86
87      li $v0, 1
88      move $a0, $s3
89      syscall
90
91      srl $t4, $s0, 3 # $t4 = A / 8

```

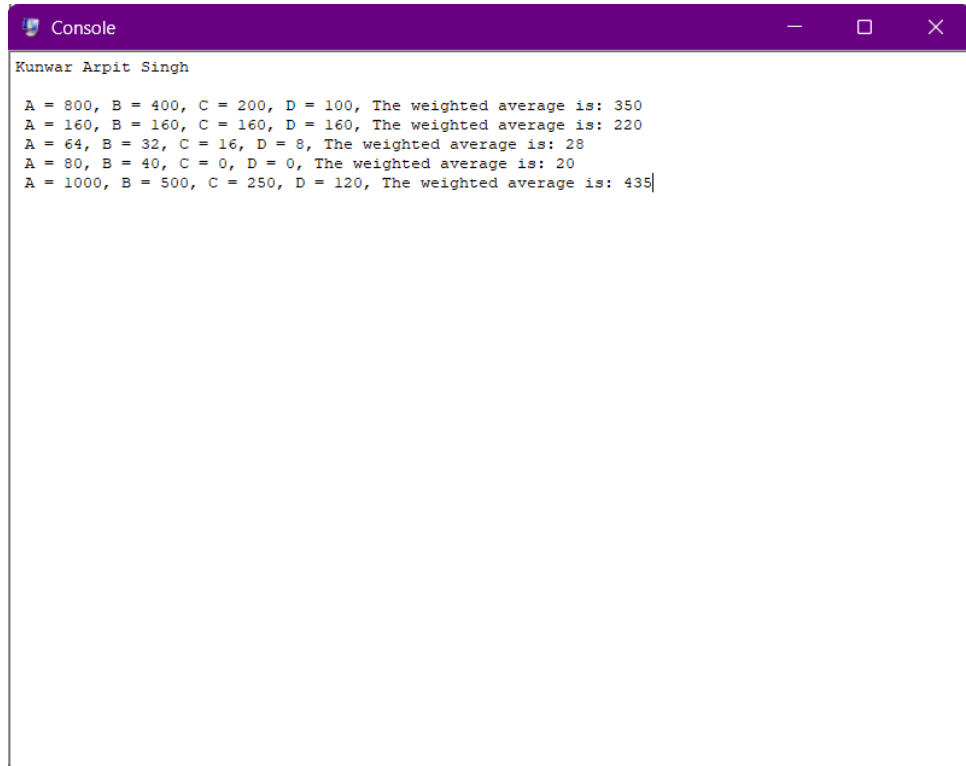
```

92      srl $t5, $s1, 2 # $t5 = B / 4
93      srl $t6, $s2, 1 # $t6 = C / 2
94      srl $t7, $s3, 1 # $t7 = D / 2
95
96      add $s4, $t4, $t5
97      add $s4, $s4, $t6
98      add $s4, $s4, $t7
99
100     li $v0, 4
101     la $a0, result_msg
102     syscall
103
104     li $v0, 1
105     move $a0, $s4
106     syscall
107
108     jr $ra

```

Output Screenshots

Console output (Weighted Average)



```

Console
Kunwar Arpit Singh
A = 800, B = 400, C = 200, D = 100, The weighted average is: 350
A = 160, B = 160, C = 160, D = 160, The weighted average is: 220
A = 64, B = 32, C = 16, D = 8, The weighted average is: 28
A = 80, B = 40, C = 0, D = 0, The weighted average is: 20
A = 1000, B = 500, C = 250, D = 120, The weighted average is: 435|

```

Register values (Weighted Average)

Int Regs [16]		
PC	=	400098
EPC	=	0
Cause	=	0
BadVAddr	=	0
Status	=	3000ff10
HI	=	0
LO	=	0
R0 [r0]	=	0
R1 [at]	=	10010000
R2 [v0]	=	a
R3 [v1]	=	0
R4 [a0]	=	1b3
R5 [a1]	=	1f4
R6 [a2]	=	fa
R7 [a3]	=	78
R8 [t0]	=	0
R9 [t1]	=	0
R10 [t2]	=	0
R11 [t3]	=	0
R12 [t4]	=	7d
R13 [t5]	=	7d
R14 [t6]	=	7d
R15 [t7]	=	3c
R16 [s0]	=	3e8
R17 [s1]	=	1f4
R18 [s2]	=	fa
R19 [s3]	=	78
R20 [s4]	=	1b3
R21 [s5]	=	0
R22 [s6]	=	0
R23 [s7]	=	0
R24 [t8]	=	0
R25 [t9]	=	0
R26 [k0]	=	0
R27 [k1]	=	0
R28 [gp]	=	10008000
R29 [sp]	=	7ffffef1c
R30 [s8]	=	0
R31 [ra]	=	400094